Assignment_3_Data_Visualization_Analytics

2026-10

Question 1

1) Load in the following files from the Data directory for Week 3:

- ALB_CR_2017-2018.xpt
- KIQ_U_2017-2018.xpt
- DEMO_2017-2018.xpt

a) After reading in the data, take a look at the head of each of these files. There is tons of information in there, but we will only be working with parts of each of these data frames. It's your job to pull data from these data frames to create a single data frame containing these columns from each of these files:

- ALB_CR_2017-2018.xpt: SEQN, URXUMA, URXUCR
- KIQ_U_2017-2018.xpt: KIQ022
- DEMO_2017-2018.xpt: RIDAGEYR, RIAGENDR

b) After creating the data frame, rename the columns using this key:

- SEQN : Subject_ID
- URXUMA: Albumin
- URXUCR: Creatinine
- RIDAGEYR: Age
- RIAGENDR: Sex
- KIQ022: Weak_Kidney (Have you ever been told you have weak or failing kidneys?)

c) You will notice that the "Sex" variable is coded with "1" for "Male" and "2" for "Female". You will also notice that "Weak_Kidney" variable is coded with "1" for "Yes", "2" for "No", "7" for "Refused", and "9" for "Don't Know". Recode these levels from the numbers to the correct categorical variables described above. Hint: Try out the `tidyverse` package and take a look at the function called `recode()`.

```
# Load the tidyverse package + ggpubr
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(ggpubr) # For the statistical tests - Wilcoxon rank sum test
library(gridExtra) # A package that helps arranging ggplot objects
```

```
##
## Attaching package: 'gridExtra'
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(haven) # load package to include read_xpt()
```

```r
# Plan
# 1. Load tidyverse and set the working directory
# 2. Read in data files.
# 3. Look at head() of each data frame to understand
#    data
# 4. Select variables needed
# 5. Merge the three data frames together on ID (seqn)
# 6. Rename the variables to requested names
# 7. Recode the numerical values for sex and weak_kidney into
#    string labels


# Set working directory to the Week 3 folder
setwd("~/Dartmouth - Foundations of Data Science - R Studio/week_3/data")


# Read albumin and creatinine lab data
lb_cr <- read_xpt("ALB_CR_2017-2018.xpt") %>%
  data.frame()

# Read kidney data
kiq_u <- read_xpt("KIQ_U_2017-2018.xpt") %>%
  data.frame()

# Read demographic data
demo <- read_xpt("DEMO_2017-2018.xpt") %>%
  data.frame()


# View head of data
head(lb_cr)
```

```
##    SEQN URXUMA URXUMS URDUMALC URXUCR  URXCRS URDUCRLC URDACT
## 1 93705    3.2    3.2        0     24  2121.6        0  13.33
## 2 93706   39.6   39.6        0     69  6099.6        0  57.39
## 3 93707   41.6   41.6        0    209 18475.6        0  19.90
## 4 93708    9.1    9.1        0     47  4154.8        0  19.36
## 5 93709   26.4   26.4        0    215 19006.0        0  12.28
## 6 93711    3.7    3.7        0     98  8663.2        0   3.78
```

```r
# View head of data
head(kiq_u)
```

```
##    SEQN KIQ022 KIQ025 KIQ026 KIQ029 KIQ005 KIQ010 KIQ042 KIQ430 KIQ044 KIQ450
## 1 93705      2     NA      2     NA      1     NA      1      1      1      1
## 2 93708      2     NA      2     NA      3      1      1      1      1      1
## 3 93709      2     NA      1      2     NA     NA     NA     NA     NA     NA
## 4 93711      2     NA      2     NA      1     NA      2     NA      2     NA
## 5 93713      2     NA      2     NA      1     NA      2     NA      2     NA
## 6 93714      2     NA      2     NA      1     NA      2     NA      2     NA
##   KIQ046 KIQ470 KIQ050 KIQ052 KIQ480
## 1      2     NA      1      1      2
## 2      2     NA      1      1      3
## 3     NA     NA     NA     NA     NA
## 4      2     NA     NA     NA      3
## 5      2     NA     NA     NA      0
## 6      2     NA     NA     NA      0
```

```r
# View head of data
head(demo)
```

```
##    SEQN SDDSRVYR RIDSTATR RIAGENDR RIDAGEYR RIDAGEMN RIDRETH1 RIDRETH3 RIDEXMON
## 1 93703       10        2        2        2       NA        5        6        2
## 2 93704       10        2        1        2       NA        3        3        1
## 3 93705       10        2        2       66       NA        4        4        2
## 4 93706       10        2        1       18       NA        5        6        2
## 5 93707       10        2        1       13       NA        5        7        2
## 6 93708       10        2        2       66       NA        5        6        2
##   RIDEXAGM DMQMILIZ DMQADFC DMDBORN4 DMDCITZN DMDYRSUS DMDEDUC3 DMDEDUC2
## 1       27       NA      NA        1        1       NA       NA       NA
## 2       33       NA      NA        1        1       NA       NA       NA
## 3       NA        2      NA        1        1       NA       NA        2
## 4      222        2      NA        1        1       NA       15       NA
## 5      158       NA      NA        1        1       NA        6       NA
## 6       NA        2      NA        2        1        7       NA        1
##   DMDMARTL RIDEXPRG SIALANG SIAPROXY SIAINTRP FIALANG FIAPROXY FIAINTRP MIALANG
## 1       NA       NA       1        1        2       1        2        2      NA
## 2       NA       NA       1        1        2       1        2        2      NA
## 3        3       NA       1        2        2       1        2        2       1
## 4       NA       NA       1        2        2      NA       NA       NA       1
## 5       NA       NA       1        1        2       1        2        2       1
## 6        1       NA       1        2        1       1        2        2       1
##   MIAPROXY MIAINTRP AIALANGA DMDHHSIZ DMDFMSIZ DMDHHSZA DMDHHSZB DMDHHSZE
## 1       NA       NA       NA        5        5        3        0        0
## 2       NA       NA       NA        4        4        2        0        0
## 3        2        2        1        1        1        0        0        1
## 4        2        2        1        5        5        0        0        1
## 5        2        2        1        7        7        0        3        0
## 6        2        1        3        2        2        0        0        2
##   DMDHRGND DMDHRAGZ DMDHREDZ DMDHRMAZ DMDHSEDZ  WTINT2YR  WTMEC2YR SDMVPSU
## 1        1        2        3        1        3  9246.492  8539.731       2
## 2        1        2        3        1        2 37338.768 42566.615       1
```

```
## 3        2        4        1        2       NA  8614.571  8338.420       2
## 4        1        4        3        1        2  8548.633  8723.440       2
## 5        1        3        2        1        3  6769.345  7064.610       1
## 6        1        4        1        1        1 13329.451 14372.489       2
##   SDMVSTRA INDHHIN2 INDFMIN2 INDFMPIR
## 1      145       15       15     5.00
## 2      143       15       15     5.00
## 3      145        3        3     0.82
## 4      134       NA       NA       NA
## 5      138       10       10     1.88
## 6      138        6        6     1.63
```

```r
merged_data <- lb_cr %>% # merge data and select requested columns
  select(SEQN, URXUMA, URXUCR) %>%
  full_join(
    kiq_u %>%
      select(SEQN, KIQ022),
    by = "SEQN" # merge on shared ID
  ) %>%
  full_join(
    demo %>%
      select(SEQN, RIDAGEYR, RIAGENDR),
    by = "SEQN"
  ) %>%
  rename( # rename variables
    Subject_ID = SEQN,
    Albumin = URXUMA,
    Creatinine = URXUCR,
    Age = RIDAGEYR,
    Sex = RIAGENDR,
    Weak_Kidney = KIQ022
  ) %>%
  mutate( # recode sex and kidney status
    Sex = recode(
      Sex,
      `1` = "Male",
      `2` = "Female"
    ),
    Weak_Kidney = recode(
      Weak_Kidney,
      `1` = "Yes",
      `2` = "No",
      `7` = "Refused",
      `9` = "Don't Know"
    )
  )

# View the first few rows of the merged data
head(merged_data)
```

```
##   Subject_ID Albumin Creatinine Weak_Kidney Age    Sex
## 1      93705     3.2         24          No  66 Female
## 2      93706    39.6         69        <NA>  18   Male
## 3      93707    41.6        209        <NA>  13   Male
```

```
## 4      93708     9.1         47          No  66 Female
## 5      93709    26.4        215          No  75 Female
## 6      93711     3.7         98          No  56   Male
```

Question 2

Create a panel including the following three plots. Make sure that the labeling is publication ready!

a) The first plot should display a violin plot of creatinine (mg/dL) levels by sex. Make sure to color the violin plots by sex. This should be a ggplot2 plot.
b) The second plot in the panel should display a scatter plot of creatinine (mg/dL) levels on the x-axis and albumin (ug/ml) levels on the y-axis. The scatter plot should have a linear regression line.
c) The final plot in the panel should be a density plot (`geom_density()`) of creatinine (mg/dL) levels by sex. Please color the density curves by sex.

```
# Plan
# 1. Create violin plot showing the distribution of creatinine value by sex, geom_jitter
 ↪  for individual points
# 2. Create scatter plot of creatinine versus albumin, fit linear regression model, and
 ↪  add the plotted line and equation to the graphic
# 3. Create a density plot showing the distribution of creatinine levels for males and
 ↪  females, setting each line to a different color
# 4. Arrange the plots into a single panel using grid.arrange() with two columns.

# Build Violin Plot

violin_plot <- ggplot(
    merged_data,
    aes(
      x = Creatinine,
      y = Sex  # Set covariate to measure by
    )
    ) +
    geom_violin( # Set theme formats
      fill = "darkgreen",
      color = "black",
      alpha = 0.4,
      scale = "width"
    ) +
    geom_jitter( # Use jitter to plot actual distinct datapoints
      width = 0.1,
      alpha = 0.2,
      size = 1
    ) +
    labs( # Set labeling
      title = paste("Creatinine Levels", "by", "Sex"),
      x = "Creatinine Levels (mg/dL)",
      y = "Sex"
    ) +
    theme_minimal(base_size = 8) +
    theme( # Set formatting for label and text
      plot.title = element_text(
        hjust = 0.5,
```

```
        face = "bold",
        size = 8
      ),
    axis.title = element_text(face = "bold"),
    axis.text.y = element_text(size = 7),
    panel.grid.minor = element_blank()
    )

# Build Scatter Plot


# Linear regression run model

model <- lm(Albumin ~ Creatinine, data = merged_data)

# Extract coefficients
coefficients <- coef(model)
intercept <- coefficients[1]
slope <- coefficients[2]

# Create equation label text
lm_label <- paste0("y = ", round(slope, 2), "x + ", round(intercept, 2))

scatter_plot <- ggplot(
    merged_data,
      aes(
        x = .data[["Creatinine"]], # Set x and y for scatter
        y = .data[["Albumin"]]
      )
  ) +
  geom_point(
      color = "darkgreen",
      alpha = 0.6 # Set transparency for better readability of data density
  ) +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE, color = "black") +
    labs(
      title = paste("Creatinine (mg/dL)", "vs", "Albumin"), # Set title
        x = "Creatinine",
        y = "Albumin"
  ) +
  theme_minimal(base_size = 8) +
  theme( # Set formatting for label and text
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"), # Bold axis titles
    panel.grid.minor = element_blank() # Remove extra gridlines
  ) +
  annotate("text", x = 20, y = 12000, label = lm_label, # set location / size after trial
   ↪  and error
           hjust = 0, vjust = 1, size = 3, color = "black")
```

```r
# Density plot

density_plot <- ggplot(
  merged_data,
  aes(
    x = Creatinine,
    color = Sex
  )
  ) +
  geom_density() + # create a density plot
  labs( # set labels
    title = "Creatinine Levels by Sex",
    x = "Creatinine Levels (mg/dL)",
    y = "Density",
    color = "Sex"
  ) +
  theme_minimal(base_size = 8) + # remove lines and set base size
  theme( # format for title and axis
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )


grid.arrange( # Put all plots on the same plane, two columns
    violin_plot,
    scatter_plot,
    density_plot,
    ncol = 2
  )
```

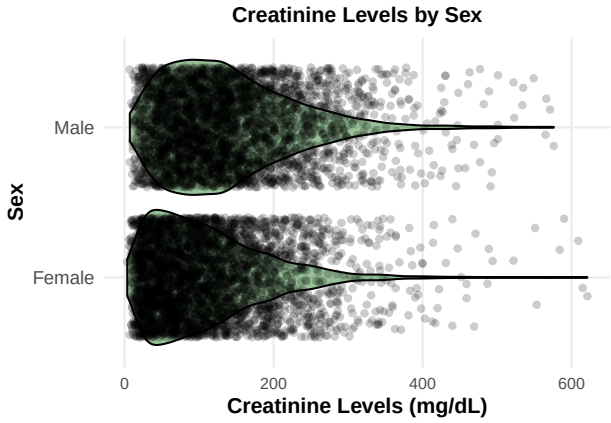
```
## Warning: Removed 1622 rows containing non-finite outside the scale range
## (`stat_ydensity()`).
```

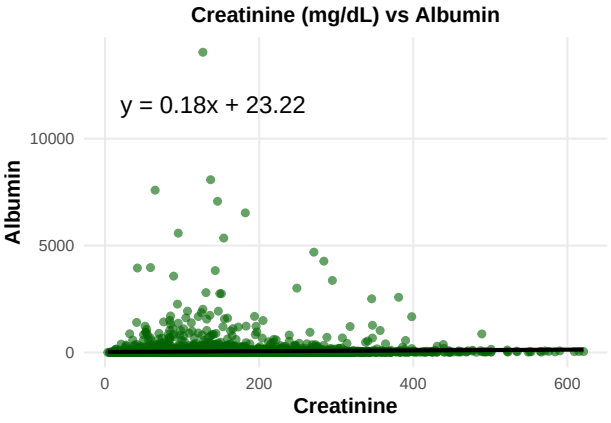
```
## Warning: Removed 1622 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

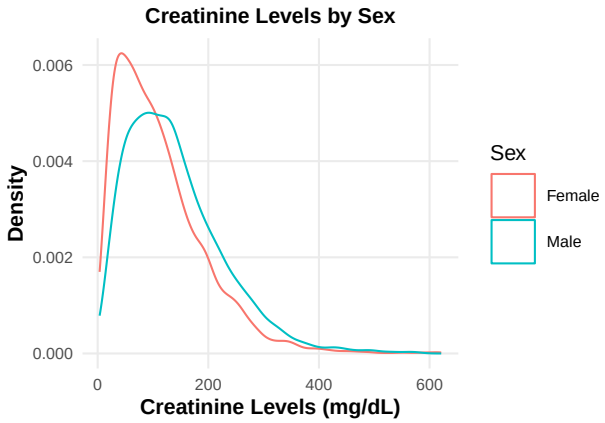
```
## Warning: Removed 1622 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 1622 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

```
## Warning: Removed 1622 rows containing non-finite outside the scale range
## (`stat_density()`).
```







Question 3

Create a function that takes the data frame above we have been working with, continuous variable, categorical variable, name of continuous variable as you would like it to appear in labels and titles, and name of categorical variable as you would like it to appear in labels and titles. The function should create a box plot of the continuous variable by the categorical variable. The plot should also display the p-value comparing the results of a Wilcoxon Rank Sum Test between the categorical variable levels. Calculate the mean and standard deviation of the continuous variable. Using the `geom_text()` function, add the mean and standard deviation to the plot below the p-value. Make sure to provide a title, appropriate labels, and colors!

```
# Plan:
# 1. Create a function that accepts a data frame, a continuous
#    variable, a categorical variable, and labels for both variables
# 2. Perform a Wilcoxon Rank Sum Test comparing the levels
#    of the categorical variable
# 3. Calculate the mean and standard deviation of the continuous
#    variable within each category
# 4. Create a box plot of the continuous variable by the
#    categorical variable
# 5. Add Wilcoxon p-value to the plot
# 6. Add mean and standard deviation below the p-value.


box_plot_function <- function(data, continuous_var, categorical_var, continuous_name,
 ↪  categorical_name) { # set inputs as requested
```

```r
# Remove NA data
plot_data <- data %>%
  filter(
    !is.na(.data[[continuous_var]]),
    !is.na(.data[[categorical_var]])
  )

# Perform wilcoxon rank sum test
wilcox_result <- wilcox.test(
  plot_data[[continuous_var]] ~ plot_data[[categorical_var]]
)

# get p value from wilcox results
p_value <- wilcox_result$p.value

# p value text to add to figure
p_label <- paste0("Wilcoxon p = ", format.pval(p_value, digits = 3))

# calculate mean and standard deviation
summary_data <- plot_data %>%
  group_by(.data[[categorical_var]]) %>%
  summarise(
    Mean = mean(.data[[continuous_var]], na.rm = TRUE),
    Std_Dev = sd(.data[[continuous_var]], na.rm = TRUE),
    .groups = "drop"
  )

# create text label for mean and standard deviation
summary_label <- paste0(summary_data[[categorical_var]],": Mean = ",
↪  round(summary_data$Mean, 2), ", Std Dev = ", round(summary_data$Std_Dev, 2), collapse
↪  = "\n")

# find the max value of the continuous variable for formatting
max_value <- max(
  plot_data[[continuous_var]],
  na.rm = TRUE
)

# put p value above plots max value so it doesn't block the plot
p_y <- max_value*1.2

# put mean below the p value - same as above
stats_y<- max_value*1.1

# Build box plot

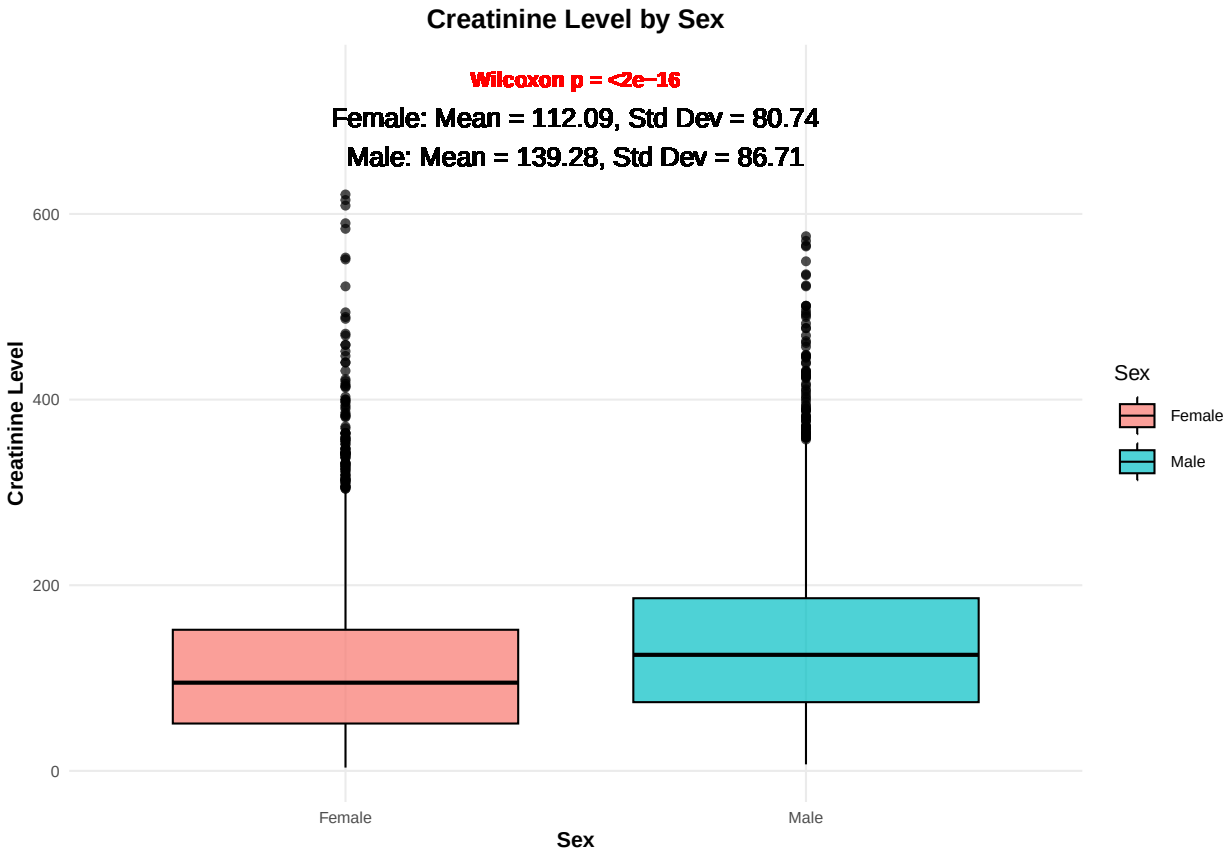
box_plot <- ggplot(
  plot_data,
  aes( # set variables
    x = .data[[categorical_var]],
    y = .data[[continuous_var]],
    fill = .data[[categorical_var]]
  )
```

```r
  ) +
  geom_boxplot(
      alpha = 0.7,
      color = "black"
    ) +
    # Add p-value
    geom_text( # add text for p value
      aes(
        x = 1.5,
        y = p_y,
        label = p_label
      ),
      color = "red",
      fontface = "bold"
    ) +
    # Add mean and standard dev
    geom_text( # add text for summ stats
      aes(
        x = 1.5,
        y = stats_y,
        label = summary_label
      ),
      size = 3.5
    ) +
    # Add title and axis labels
    labs(
      title = paste(
        continuous_name,
        "by",
        categorical_name
      ),
      x = categorical_name,
      y = continuous_name,
      fill = categorical_name
    ) +
    theme_minimal(base_size = 8) + # set minimal theme and default size
    theme( # set title formatting
      plot.title = element_text(
        hjust = 0.5,
        face = "bold",
        size = 10
      ),
      axis.title = element_text(
        face = "bold" # set axis as bold
      ),
      panel.grid.minor = element_blank() # remove gridlines
    )

  # Return plot
  return(box_plot)
}

# generate boxplot using functioj
box_plot <- box_plot_function(merged_data, "Creatinine", "Sex", "Creatinine Level",
 ↪  "Sex")
```

```
box_plot
```



Question 4

Using a for loop, create 5 random data frames containing the following columns:

- Participant ID: There should be 100 participants in each data frame with unique identifiers.
- Sex: Sex should be comprised of two levels - Male and Female
- Age: This data frame should contain only adults between 18 and 85.
- Red Blood Cell Count: Female: 3.92 - 5.13 cells/L (trillion), Male: 4.35-5.65 cells/L (trillion)
- White Blood Cell Count: 3.4-9.6 cell/L (billion)

Put these data frames into a list.

```
# Plan
# 1. Set number of observations and initialize blank list
# 2. Loop through a for loop five times, building a data frame each time
#.   generating random data across the dataframe as requested.
# 3. Add each random data frame to the blank list in each itearation of the loop

n <- 100 # set number of observations to build out
```

```r
# Create an empty list to store the 5 data frames
blood_cell_data <- list()

# Loop 5 times
for (i in 1:5) { # loop over the data generation 5 times
  random_data <- data.frame(
    Participant_ID = paste("pt", 1:n, sep = "_"),  # Generate participant ids
    Sex = sample(c("Male", "Female"), n, replace = TRUE),  # Random sex values
    Age = pmax(18, pmin(85, round(rnorm(n, mean = 50, sd = 10)))),  # Random ages between
    ↪  18 and 85
    white_blood_cell = runif(n, min = 3.4, max = 9.6)  # Random WBC on uniform dist over
    ↪  range
  ) %>%
  group_by(Sex) %>% # Group by the sex variable
  # create uniform dist over range for RBC based on sex
  mutate(
    red_blood_cell = if (cur_group()$Sex == "Male") {
      runif(n(), min = 4.35, max = 5.65)
    } else {
      runif(n(), min = 3.92, max = 5.13)
    }
  )
  blood_cell_data[[i]] <- random_data # use the index variable to add dataframe to my
 ↪  list
}
```

Question 5

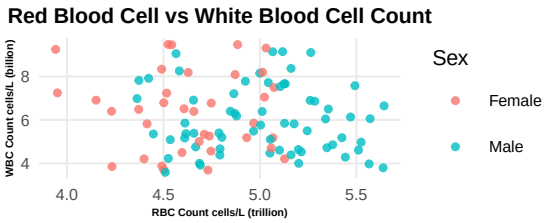
For each of the data frames you generated in the previous question, create a scatter plot with the red blood cell count on the x-axis and the white blood cell count on the y-axis. Color the points by Sex. Once you have created these plots, arrange the plots in a panel using `ggarrange()`. Hint: Make a list of the plots that you can apply to the function `do.call()` along with `ggarrange()`!

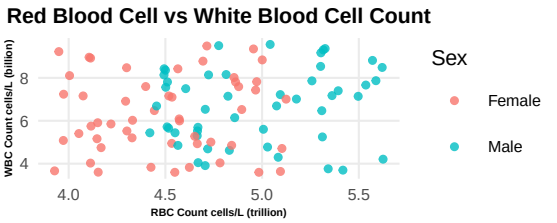
```r
# Plan
# 1. Use lapply to apply my list of dataframes to a function that plots scatter plots
# 2. Use do.call() to apply the list of the scatter plots within the ggarrange function
 ↪  to make a panel of plots

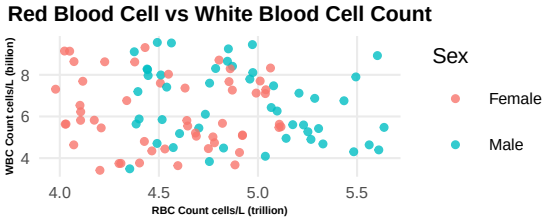
# write scatter plot
scatter_plots_blood_cells <- lapply( # apply list to function
  blood_cell_data,
  function(data) {
    ggplot(
      data,
      aes(
        x = red_blood_cell,
        y = white_blood_cell,
        color = Sex # set grouped coloring
      )
    ) +
      geom_point(
        alpha = 0.8 # set transparency and add points for scatter
      ) +
```

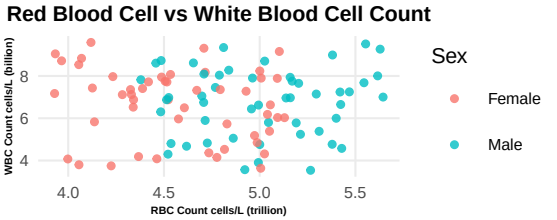
```
      labs( # set labels for x y and title
        title = "Red Blood Cell vs White Blood Cell Count",
        x = "RBC Count cells/L (trillion)",
        y = "WBC Count cells/L (billion)",
        color = "Sex"
      ) +
      theme_minimal(base_size = 8) + # set default size and minial theme
      theme( # set title formatting
        plot.title = element_text(
          hjust = 0.5,
          face = "bold",
          size = 8
        ),
        axis.title = element_text( # set axis formatting
          face = "bold",
          size = 4
        ),
        panel.grid.minor = element_blank(), # remove grid lines
        legend.text = element_text(size = 6),
        legend.title = element_text(size = 8)
      )
  }
)

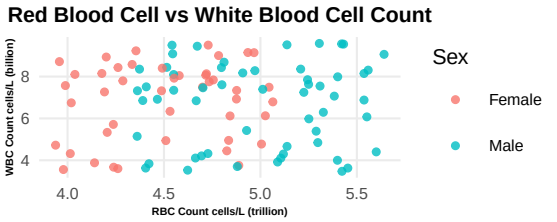
do.call( # apply ggarrange() across the list of scatter plots
  ggarrange,
  c(scatter_plots_blood_cells, ncol = 2, nrow = 3,spacing = 1), # add spacing for no
   ↪  overlap
)
```


Red Blood Cell vs White Blood Cell Count
WBC Count cells/L (billion)
8
6
4
4.0
4.5
5.0
5.5
RBC Count cells/L (trillion)
Sex
Female
Male


Red Blood Cell vs White Blood Cell Count
WBC Count cells/L (billion)
8
6
4
4.0
4.5
5.0
5.5
RBC Count cells/L (trillion)
Sex
Female
Male


Red Blood Cell vs White Blood Cell Count
WBC Count cells/L (billion)
8
6
4
4.0
4.5
5.0
5.5
RBC Count cells/L (trillion)
Sex
Female
Male


Red Blood Cell vs White Blood Cell Count
WBC Count cells/L (billion)
8
6
4
4.0
4.5
5.0
5.5
RBC Count cells/L (trillion)
Sex
Female
Male


Red Blood Cell vs White Blood Cell Count
WBC Count cells/L (billion)
8
6
4
4.0
4.5
5.0
5.5
RBC Count cells/L (trillion)
Sex
Female
Male